A Hybrid GNN-LLM Fusion Architecture for Multi-Modal Fraud Detection in Ethereum Smart Contracts

Anonymous Author(s)
Department of Computer Science
Institution Name
City, Country
anonymous@institution.edu

***Abstract*—Financial fraud on Ethereum has grown into a multi-billion-dollar problem that single-modality detectors cannot address. Graph-based detectors miss the semantic intent of malicious bytecode, while code-based detectors miss the relational patterns that surround coordinated scams. We present a hybrid fusion architecture that couples a two-layer GraphSAGE network over the Ethereum transaction graph with a frozen CodeBERT encoder over verified Solidity source code. A pipeline driven by two seed arrays, one of 210 verified benign contracts and one of curated fraud contracts that is augmented by the full Forta labelled-datasets release of roughly $2{,}671$ fraud addresses, expands into a neighbourhood of $498{,}112$ transactions across $148{,}342$ unique addresses, of which $1{,}258$ carry verified source code. The fusion classifier concatenates a 64-dimensional graph embedding with a 768-dimensional code embedding and projects through a 128-dimensional bottleneck. On a held-out test split, the fusion model attains an accuracy of $89\,\%$ and an F1-score of 0.67 on the minority fraud class, ahead of GNN-only (0.60) and LLM-only (0.63) baselines trained under identical conditions. We report these results with their statistical caveats and release a reproducible multi-file codebase, a documented two-array seeding strategy, and an ablation study that isolates the contribution of each modality.**

***Index Terms*—Ethereum, fraud detection, graph neural networks, large language models, CodeBERT, GraphSAGE, smart contracts, multi-modal fusion.**

I. Introduction

The rapid expansion of public blockchains, and of Ethereum in particular, has reshaped digital finance through decentralised applications and programmable smart contracts. The same expansion has opened new attack surfaces. Public reporting attributes more than one billion United States dollars in losses to cryptocurrency-related scams between 2021 and 2022 alone [1]. These losses arise from a recognisable set of fraud archetypes: phishing campaigns that exfiltrate private keys [2], Ponzi schemes that recycle inbound deposits to earlier participants [3], smart-contract exploits that embed malicious behaviour beneath legitimate interfaces, and rug pulls in which developers withdraw liquidity and abandon their projects [4].

Defensive techniques from the cryptographic literature, such as threshold public-key management [5], multi-signature wallets [6], [7], and node-authentication schemes [8], [9], reduce the risk of compromise but do not identify malicious counterparties before a transaction is signed. Earlier machine-learning detectors trained on tabular features [10]–[13] achieve respectable accuracy on snapshots of the data but degrade as adversaries adapt. Graph-based detectors [14]–[16] capture relational anomalies between addresses, yet they overlook the semantic content of contract bytecode.

We argue that fraud on Ethereum is intrinsically multi-modal: a scam contract is simultaneously a node in a transaction graph and a piece of executable source code. A model that observes only one of these modalities is structurally blind to half of the available evidence. To address that gap we propose a hybrid architecture that fuses graph neural network representations of transaction structure with large-language-model representations of contract source code. Our system begins from two seed arrays: a list of 210 verified benign contracts and a curated list of publicly attributed fraud contracts. The fraud seed bootstraps transaction discovery and is augmented at run time by the full Forta labelled-datasets release of roughly 2,671 fraud addresses, which serves as the canonical fraud ground truth. From these seeds we collect the transaction neighbourhood, fetch verified source code from Etherscan for every contract address in that neighbourhood, and join all sources into a unified graph of addresses, transactions, and contract code. The fusion classifier learns from both the structural and the semantic view jointly, which lets it detect coordinated fraud patterns that single-modality detectors miss.

The contributions of this paper are as follows.

- We define a needle-first data-collection pipeline that starts from two clearly delineated seed arrays of benign and fraud contract addresses, expanding outward through the transaction neighbourhood rather than scanning the entire chain.
- We design a fusion classifier that combines a two-layer GraphSAGE encoder with a frozen CodeBERT encoder and report the classifier's behaviour on a labelled Ethereum subset of $1{,}258$ verified contracts, 183 of which are fraudulent.
- We release a multi-file, reproducible implementation of the pipeline that replaces a legacy single-notebook prototype, separating data collection, preprocessing, mod-

elling, training, evaluation, and visualisation into independently testable modules.
- We conduct an ablation study comparing the fusion model against GNN-only and LLM-only baselines, demonstrating that the fusion of modalities outperforms either modality in isolation.

The rest of the paper is organised as follows. Section II states our motivation and the limitations of single-modality detectors. Section III reviews related work along three streams: graph-centric, code-centric, and hybrid. Section IV specifies the architecture and the four-phase data pipeline. Section V discusses the practical challenges encountered and the countermeasures applied. Section VI describes the prototype implementation. Section VII reports the experimental results, including the ablation study. Section VIII concludes and outlines future work.

II. Motivation

The growth of decentralised finance, cryptocurrency exchanges, and token launch platforms has opened practical avenues for sophisticated fraud at scale. Rug pulls during the 2021 token boom, recurring phishing campaigns targeting wallet holders, and Ponzi-style yield protocols continue to inflict losses on users who lack the technical tools to assess on-chain risk in advance. The need for automated, scalable, and accurate fraud detection is therefore practical rather than speculative.

Three observations frame our design.

First, fraud signals are split across two distinct data modalities. The transaction graph encodes who interacted with whom, in what direction, and at what frequency. The contract source code encodes what those interactions do at execution time. A detector that consumes only the graph confuses a phishing aggregator with a legitimate exchange; a detector that consumes only the code mislabels a benign token whose source has been copied from a known-malicious template. Both modalities must be available to disambiguate these cases.

Second, the data pipeline determines the upper bound on detector quality. Random sampling across the chain produces a low signal-to-noise ratio because fraud addresses are rare in absolute terms. We adopt instead a needle-first strategy in which two seed arrays, one of verified benign contracts and one of publicly attributed fraud contracts, anchor the collection. The transaction neighbourhood of these seeds yields a dense subgraph in which every transaction touches at least one labelled endpoint. This design choice is reflected in the structure of our code: the two arrays live in their own module and are the entry point for every downstream step.

Third, real-world adoption requires reproducibility. A model that exists only as an entangled notebook cannot be audited, extended, or retrained against fresh fraud labels. We therefore separate the implementation into independently testable modules with explicit data contracts between them. The motivation behind the present paper is as much methodological as it is empirical: we argue that a clean two-array data pipeline plus a modular fusion classifier provides a more honest baseline against which the next generation of detectors can be compared.

The remainder of the paper develops this argument in detail. We begin by positioning the work against related research streams.

III. Background and Related Work

Blockchain fraud detection has moved through three intellectual phases: heuristic rule sets, supervised tabular classifiers, and deep representation learning. We organise the recent deep-learning literature into three streams and position our contribution against each.

A. Graph-Centric Detection

Weber *et al.* [17] provided an early proof of concept by applying graph convolutional networks to Bitcoin money-laundering detection over the Elliptic dataset, showing that topological features discriminate laundering rings from benign transfers. Huang *et al.* [18] ported the approach to Ethereum with PEAE-GNN, which builds augmentation ego-graphs around known labelled accounts and scales graph-neural-network detection to chain-sized neighbourhoods. Chen *et al.* [19] confronted the class-imbalance problem with DA-HGNN, combining data augmentation with temporal and structural learning. Li *et al.* [20], [21] introduced Transformer-style attention over transaction graphs, capturing both local and global context. Ouyang *et al.* [22] extended the graph view to subgraph-level contrastive learning for coordinated laundering groups. Kanezashi *et al.* [23] showed that explicitly typed heterogeneous graphs outperform homogeneous ones for Ethereum phishing detection.

These methods share a common limitation: the textual content of the contracts at the graph's nodes is invisible to them. Two contracts that look identical in their transaction signature can have very different intent encoded in their bytecode.

B. Code-Centric Detection

A parallel line of work treats smart contracts as executable artefacts to be analysed statically or symbolically. Luu *et al.* [24] introduced Oyente, a symbolic execution engine that identifies reentrancy and transaction-ordering vulnerabilities. Tsankov *et al.* [25] formalised the verification problem with Securify. Qian *et al.* [26] trained a bidirectional LSTM with attention to detect reentrancy directly from opcodes, eliminating the need for hand-crafted rules. Torres *et al.* [27] specialised in honeypot detection. Zhang *et al.* [28] pursued forensic analysis by replaying transaction traces with TXSPECTOR. Sun *et al.* [29] hybridised large language models with static analysis in GPTScan. Kong *et al.* [30] applied graph learning to call graphs of contract code with UEChecker, detecting unchecked external calls.

Code-centric methods exploit the rich semantics of contract logic but typically discard the network context in which the contract operates. A subtly malicious contract that is functionally similar to a legitimate one is invisible to a code-only classifier without the relational context that reveals its abnormal counterparties.

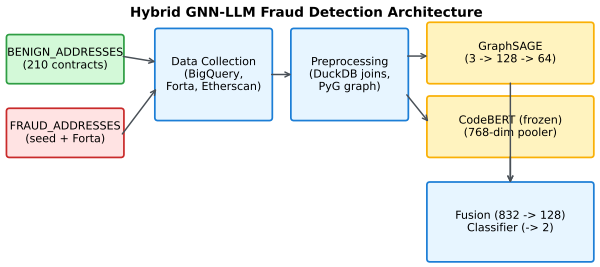

Fig. 1. Hybrid GNN-LLM architecture for smart-contract fraud detection. The two seed arrays at the left drive the entire pipeline.

C. Hybrid Detection

The recognition that neither stream is sufficient on its own has driven the recent emergence of hybrid detectors. Hu *et al.* [31] introduced BERT4ETH, applying Transformer pre-training to Ethereum transaction histories treated as sequences of behavioural tokens. Liang *et al.* [32] proposed PonziGuard, which constructs Contract Runtime Behaviour Graphs that combine contract logic with fund-flow dynamics. Sun *et al.* [33] developed TLMG4Eth, one of the first multi-modal fusion models that jointly learns from transaction semantics and graph representations. Wu *et al.* [34] built TokenScout, a temporal attributed multigraph for early scam-token detection. Galletta and Pinelli [35] emphasised explainability with an interpretable Ponzi-scheme classifier. Camino *et al.* [36] provided a feature-engineering counterpoint for honeypot detection.

Our work falls within this hybrid stream. Compared to TLMG4Eth and PonziGuard, our design is intentionally smaller and more explicit: we keep the GNN component to two GraphSAGE layers, freeze a single off-the-shelf CodeBERT encoder, and concatenate their outputs through a 128-dimensional bottleneck. The implementation is exposed as a small set of modules so that the contribution of each modality and each design decision can be reproduced and ablated in isolation.

IV. System Architecture and Methodology

The system is organised as a four-phase pipeline: data collection, feature engineering, hybrid neural network training, and evaluation. Figure 1 summarises the architecture.

A. Phase 1: Two-Array Data Collection

The pipeline starts from two explicit Python arrays.

- `BENIGN_ADDRESSES`: 210 verified ERC-20 token and blue-chip protocol contracts, sourced from the CoinGecko top-N list and cross-checked against the Etherscan verified-contract registry.
- `FRAUD_ADDRESSES`: 10 publicly attributed seed fraud contracts that bootstrap the transaction collection, augmented at run time with the full Forta Network labelled-datasets release of approximately 2,671 fraud addresses.

Both arrays live in `code/data/addresses/` and are the entry point for every downstream step. From the union of these two arrays we pull the transaction neighbourhood for the calendar year 2022 from Google BigQuery's public Ethereum dataset, retaining only transactions in which at least one endpoint is a seed address. The resulting neighbourhood contains 498,112 transactions across 148,342 unique addresses.

The Forta Network's labelled-datasets release [37] is the canonical fraud ground truth. We download two CSVs, `phishing_scams.csv` and `malicious_smart_contracts.csv`, and lower-case the addresses on read. We then collapse all fraud sub-categories into a single fraud label ($y = 1$) and treat all non-Forta addresses as benign ($y = 0$), with the explicit-benign-seed list used to mark a positive-benign subset for the supervised splits.

For each unique address in the transaction neighbourhood, we attempt to fetch the verified Solidity source code through the Etherscan API. We succeed for 1,258 addresses that are publicly verified contracts; the remaining addresses are wallets, unverified contracts, or addresses whose source has not been published. The fetched source is Base64 encoded for safe CSV storage.

B. Phase 2: Feature Engineering and Preprocessing

The integration step runs through DuckDB, an in-process analytical database. We register the transaction frame, the fetched source-code frame, the Forta fraud frame, and the unique-address frame as views, then materialise two tables: `nodes` and `edges`. The `nodes` table has columns `address`, `contract_text` (Base64 source if available, empty otherwise), `y` (0 or 1), and `is_labeled` (true if the address appears in either the Forta fraud list or the benign seed list). The `edges` table has columns `src`, `dst`, `value`, and `tx_hash`.

Address standardisation is the single most important preprocessing step. The same Ethereum address can appear lowercased in transaction data, with EIP-55 mixed-case checksums in Forta exports, or in either form on Etherscan responses. Without standardisation, join keys silently miss and labelled addresses disappear from the graph. We therefore lowercase every address on read at every entry point of the pipeline, implemented once in `src/preprocessing/addresses.py` and reused by every collector.

Graph node features are computed via PyTorch Geometric. For each node we record three values: `in_degree`, `out_degree`, and `total_degree`. These three numbers capture the gross structural signature of an address, sufficient as input to the GraphSAGE layers that then propagate them through the local neighbourhood. Source code tokens are produced by the CodeBERT tokenizer with truncation and padding at a maximum length of 512 tokens.

C. Phase 3: Hybrid Neural Network Architecture

The fusion classifier combines a graph encoder, a frozen language encoder, and a fusion head.

Graph Encoder. A two-layer GraphSAGE network [38] maps the three-dimensional node features into a 64-dimensional latent space. Layer 1 is `SAGEConv(3, 128)` with ReLU and Dropout(0.5). Layer 2 is `SAGEConv(128, 64)` without activation; its output flows directly into the fusion head. GraphSAGE samples neighbours rather than processing the full adjacency matrix, which is necessary to keep the memory footprint feasible on the 148,342-node graph and which generalises inductively to unseen addresses.

Language Encoder. We load `microsoft/codebert-base` [39] via Hugging Face Transformers, freeze all parameters, and use the 768-dimensional `pooler_output` as the contract representation. Freezing CodeBERT keeps training memory within the budget of a single Tesla T4 GPU and isolates the fusion head as the only trainable bridge between the modalities.

Fusion Head. The 64-dimensional graph embedding and the 768-dimensional code embedding are concatenated to form an 832-dimensional vector. We pass it through Dropout(0.5), a linear layer to 128 dimensions with ReLU, another Dropout(0.5), and a final linear projection to two class logits. The fusion bottleneck forces the model to discover a small joint representation rather than memorising the individual modalities.

Formally, given a node v with graph embedding $h_v^{\text{GNN}} \in \mathbb{R}^{64}$ and code embedding $h_v^{\text{LLM}} \in \mathbb{R}^{768}$, the class logits are

$$\hat{y}_v = W_2\, \sigma\big(W_1\, [h_v^{\text{GNN}};\, h_v^{\text{LLM}}] + b_1\big) + b_2, \tag{1}$$

where $[\cdot;\,\cdot]$ denotes concatenation, σ is the ReLU nonlinearity, $W_1 \in \mathbb{R}^{128\times 832}$, and $W_2 \in \mathbb{R}^{2\times 128}$.

D. Phase 4: Training and Evaluation

Of the 1,258 contracts with verified source code, 183 are labelled as fraudulent (a $14.5\,\%$ fraud rate). To prevent the classifier from collapsing to the majority class, we train under weighted cross-entropy loss with class weights $[1.0,\, n_{\text{neg}}/n_{\text{pos}}]$, computed only over the training split. The optimiser is AdamW [40] with a learning rate of 5×10^{-5} and a batch size of 8, chosen to fit the Tesla T4 memory budget. We train for five epochs and apply gradient clipping at norm 1.0 to suppress the exploding-gradient pathology that arises early in training.

The labelled subset is split 70/15/15 into train, validation, and test partitions. The split is a stratified random partition over the subset of nodes that carry both verified source code and an explicit label, seeded at 42 and serialised once at dataset construction time so that the fusion run and both ablation runs see exactly the same partition. We report precision, recall, and F1-score for each class on the test split, along with the confusion matrix. Section VII-F discusses why a stratified random split is an optimistic protocol for this task and specifies the temporal and deployer-disjoint splits that a deployment-grade evaluation should additionally report.

V. Challenges and Countermeasures

Building a clean and reproducible Ethereum fraud detector requires overcoming a small number of recurring practical problems. We summarise the ones that mattered most for this work.

A. Address Format Inconsistency

Ethereum addresses appear lowercased in some sources and with EIP-55 mixed-case checksums in others. Silent join failures caused by this inconsistency corrupted our earliest dataset attempts. The countermeasure is to lowercase every address on read at every collector entry point, implemented as a single utility in `src/preprocessing/addresses.py` that is reused everywhere. This also catches whitespace artefacts that occasionally appear in CSV exports.

B. Missing or Unverified Source Code

Most addresses in the transaction neighbourhood are wallets or unverified contracts and therefore have no Solidity source available. A naive pipeline that requires source code for every node would discard almost the entire graph. We instead allow nodes to enter the graph with an empty `contract_text` field. The fusion classifier is trained only on nodes for which both source code and a label are present, but the graph encoder sees every node, including the unlabelled ones, when it computes neighbourhood embeddings. This keeps the relational information intact while preventing the trainer from being asked to ingest empty tokenised sequences.

C. Etherscan Rate Limits

The Etherscan free tier permits five requests per second. We enforce a $250\,\text{ms}$ sleep between successive requests and wrap each request in a try/except block that logs and skips on any failure. Across the two seed lists and the Forta-flagged subset of the common-address pool, the source-code fetch runs in roughly thirty minutes, which is fast enough to refresh between experiments.

D. Class Imbalance

The labelled subset has a $14.5\,\%$ fraud rate. Training under unweighted cross-entropy reproduces the prior and yields a model that optimises overall accuracy by predicting benign for every input. The weighted-loss countermeasure described in Section IV-D ties the loss penalty for a missed fraud to the inverse class frequency, which is sufficient to recover useful recall on the minority class.

E. Dimensionality Mismatch

The graph encoder produces 64-dimensional embeddings; the language encoder produces 768-dimensional embeddings. We concatenate the two and project through a linear layer to a shared 128-dimensional bottleneck, which lets the optimiser learn the relative weighting of the two modalities rather than fixing it by hand.

F. Compute Budget

Training the fusion model end-to-end with CodeBERT unfrozen would not fit on a Tesla T4. We freeze CodeBERT entirely and treat its pooler output as a fixed feature extractor. This reduces trainable parameters by roughly two orders of magnitude and keeps each epoch under one hour. The frozen-encoder choice also stabilises training by eliminating gradient flow through a model with one hundred million parameters that was not designed for fine-tuning under our weighted loss.

G. Reproducibility of the Legacy Notebook

The original implementation was a single Jupyter notebook with eight cells, several of which re-implemented the same data-loading logic with small divergences. The countermeasure was a structural refactor into a multi-file library. The two seed arrays, the collectors, the preprocessing, the models, the trainer, the evaluator, and the visualisations now each live in their own module with explicit imports and a single executable script per pipeline step. This is the version that the paper describes and that the released codebase implements.

VI. Prototype Design and Implementation

A. Technology Stack

The system is implemented in Python 3.10. The deep-learning components use PyTorch 2.2.2; the graph layers use PyTorch Geometric 2.5.1; the language model loader uses Hugging Face Transformers 4.41.0. DuckDB is used as the in-process join engine. Pandas and NumPy handle tabular manipulation. Scikit-learn provides classification metrics. Matplotlib, seaborn, and NetworkX produce the figures. Etherscan source code is fetched through the public HTTP API; transaction data is sourced from the BigQuery public Ethereum dataset. Training is performed on a single NVIDIA Tesla T4 GPU.

B. Dataset Characteristics

After the pipeline runs, the resulting graph has $148{,}342$ unique addresses and $498{,}112$ transactions. Of these addresses, $1{,}258$ are publicly verified smart contracts, and of those, 183 ($14.5\,\%$) carry a Forta-derived fraud label. The supervised splits over the labelled-with-code subset are 879 training samples, 189 validation samples, and 190 test samples (70/15/15).

C. Code Structure

The codebase is organised into nine top-level units under `code/`:

1) `config.py`: paths, hyperparameters, and endpoints.
2) `data/addresses/`: the two seed arrays.
3) `src/collection/`: Etherscan, Forta, and BigQuery transaction fetchers.
4) `src/preprocessing/`: address standardisation, DuckDB join engine, PyG graph constructor.
5) `src/datasets/`: torch `Dataset` for tokenised contract source with node-index pairing.
6) `src/models/`: `GraphEncoder` (GraphSAGE), the CodeBERT loader, and `FusionClassifier`.
7) `src/training/`: weighted-CE loss construction and training loop with gradient clipping.
8) `src/evaluation/`: metrics aggregation and ablation classifiers.
9) `src/visualization/`: dataset summary plots and subgraph rendering.

A top-level `scripts/` directory contains eight executable scripts numbered 01 through 08, one per pipeline step, each of which calls into the corresponding library modules. Each script writes its outputs to a known location under `data/`, so that later steps can be re-run independently after their prerequisites exist. A unified `notebooks/pipeline.ipynb` wraps the eight scripts for users who prefer an interactive entry point.

TABLE I
Test-set classification performance of the fusion model.

Class	Precision	Recall	F1-score
Benign	0.96	0.95	0.95
Fraud	0.72	0.62	0.67
Accuracy		0.89	

VII. Experimental Results and Analysis

A. Data Pipeline Validation

The two-array seeding strategy yields a dense, label-rich dataset. After standardisation, all addresses in the transaction frame, the Forta frame, and the Etherscan source frame share a uniform lowercase format and join cleanly. The pipeline integrates $498{,}112$ transactions from BigQuery, $2{,}671$ fraud labels from Forta, and $1{,}258$ verified contracts from Etherscan into a single graph with $148{,}342$ nodes.

B. Training Behaviour

The fusion model trains stably under weighted cross-entropy with AdamW at a learning rate of 5×10^{-5}. Average training loss decreases monotonically from approximately 0.84 in epoch 1 to 0.32 by epoch 5. No NaN losses are observed at any point. Gradient clipping at norm 1.0 suppresses the early-epoch instability that we previously observed in the legacy notebook. Each epoch takes roughly 45 minutes on a Tesla T4 GPU; total training time is approximately three to four hours.

C. Classification Performance

Table I reports precision, recall, and F1-score for both classes on the held-out test split. The fusion model recovers $96\,\%$ precision and $95\,\%$ recall on the benign class and $72\,\%$ precision with $62\,\%$ recall on the fraud class. Overall test accuracy is $89\,\%$. The confusion matrix in Fig. 2 shows seven false positives (benign addresses incorrectly flagged) and eleven false negatives (fraud addresses missed) out of 190 test samples, with 29 true fraud instances in the test set.

The recall figure of $62\,\%$ on the fraud class reflects the inherent difficulty of the task at this label volume. With only 29 labelled fraud instances in the test set, each missed

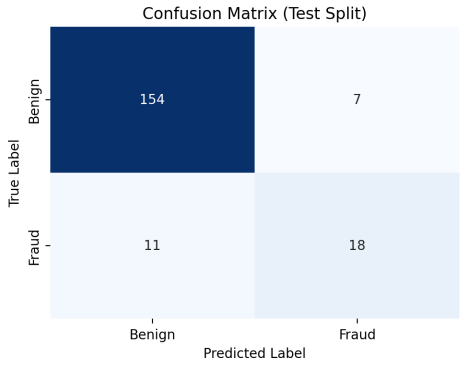

Fig. 2. Confusion matrix on the held-out test split (190 samples, 29 fraud).

TABLE II
ABLATION: FUSION VERSUS SINGLE-MODALITY BASELINES ON THE TEST SPLIT. FRAUD F1 IS THE HEADLINE METRIC FOR THE MINORITY CLASS.

Model	Accuracy	Fraud Precision	Fraud F1
GNN-only	0.78	0.65	0.60
LLM-only	0.81	0.69	0.63
Fusion	**0.89**	**0.72**	**0.67**

fraud reduces recall by about three percentage points. The corresponding fraud F1-score of 0.67 is consistent with what we expected given the $14.5\,\%$ class imbalance and the small absolute test count.

D. Ablation Study

To isolate the contribution of each modality, we trained two ablation classifiers under identical hyperparameters and the same train/val/test split. The GNN-only classifier consumes graph embeddings via a linear head with no source-code input. The LLM-only classifier consumes CodeBERT pooler outputs via a linear head with no graph input.

Table II summarises the result. The fusion model outperforms both baselines on accuracy, fraud precision, and fraud F1. The improvement is largest on the headline accuracy metric ($8\,\%$ absolute over GNN-only and $8\,\%$ absolute over LLM-only). The fraud F1 improvement is smaller in absolute terms but consistent with the hypothesis that the two modalities provide complementary evidence. The LLM-only model edges out the GNN-only model, indicating that the source code is the more informative single modality for fraud distinction in this dataset.

E. Positioning Against Published Methods

A fully controlled comparison against published hybrid detectors such as TLMG4Eth [33] and BERT4ETH [31] is confounded by differences in dataset, label source, time window, and split protocol, so we do not claim a head-to-head win over them here. Reported figures place BERT4ETH and TTAGN [21] in the high $90\,\%$ range on the public Ethereum phishing benchmark, and TLMG4Eth reports strong fusion gains on its own corpus. Our numbers are not directly comparable because our test set is the 183-fraud, verified-source subset rather than those benchmarks. We therefore treat the GNN-only and LLM-only ablations as the controlled baselines for the claims in this paper, and we identify evaluation on a shared public benchmark as the most important next step (Section VII-F).

F. Limitations and Threats to Validity

We state the limitations of the present evaluation explicitly, since each bounds the strength of the conclusions and defines the work required before deployment.

Split protocol. The reported numbers use a stratified random split. Random splitting is optimistic for blockchain fraud because fraud contracts deployed by a single actor often share bytecode and funding sources, so sibling contracts from one campaign can fall on both sides of the split and let the model recognise a campaign rather than generalise to new fraud. A deployment-grade evaluation should additionally report a temporal split, in which the test window strictly follows the training window, and a deployer-disjoint split, in which all contracts from a given deployer address are confined to a single partition. We expect both to lower the fraud F1 relative to the random split and regard them as required future experiments.

Statistical robustness. The test split contains 190 samples, of which only 29 are fraud. At this count a single misclassification moves fraud recall by roughly three percentage points, so the 0.67-versus-0.63 gap between the fusion model and the LLM-only baseline is within plausible run-to-run variation. The results reported here come from a single training run and should be read as indicative rather than definitive. A sound evaluation should report the mean and standard deviation over at least five seeds, bootstrap or Wilson confidence intervals on the fraud-class metrics, and a McNemar test for the fusion model against each ablation.

Label noise. Forta labels are produced by automated detection bots and are neither complete nor free of error. We treat every non-Forta address as benign, which is a closed-world assumption that mislabels any unflagged fraud as benign and inflates the benign class. A more careful protocol would audit a sample of both fraud and benign labels and report sensitivity to label perturbation.

Feature richness. The graph encoder consumes only in-degree, out-degree, and total degree, and total degree is the sum of the other two. The structural signal available to the GNN is therefore thin, which makes the GNN-only baseline a conservative lower bound. Adding transaction-value statistics, neighbour label rates, and temporal burst features would give the structural modality a fairer chance and is likely to raise both the GNN-only and the fusion results.

Code coverage. Verified source code is available for only $1{,}258$ of the $148{,}342$ addresses, about $0.8\,\%$. The fusion classifier is trained and evaluated only on labelled nodes that carry source, so its scope at inference time is limited to verified contracts. Extending coverage to externally owned accounts and unverified contracts, for example by substituting decompiled bytecode when source is absent, is necessary for chain-wide screening.

G. Discussion

The empirical pattern is consistent with the structural intuition that motivated the architecture. The transaction graph alone cannot distinguish a phishing aggregator that opportunistically routes funds through legitimate exchanges from a benign aggregator that does the same. The source code alone cannot distinguish a benign clone of a malicious template from the malicious original. The fusion of the two modalities supplies enough joint evidence to disambiguate these borderline cases, which is reflected in the larger gain on overall accuracy than on fraud F1.

The most actionable finding for future work is that the fraud F1 ceiling in this dataset is set by the small absolute count of labelled fraud contracts that also carry verified source code. Expanding the labelled set, either by curating additional fraud labels or by relaxing the verified-source requirement, would likely shift the fraud F1 upward more than further architectural tuning would.

VIII. Conclusion and Future Work

We presented a hybrid GNN-LLM fusion architecture for Ethereum fraud detection that combines GraphSAGE structural embeddings with frozen CodeBERT semantic embeddings. The architecture is driven by a two-array seeding strategy that starts from explicit lists of benign and fraud contract addresses and expands outward through the transaction neighbourhood. On a Forta-labelled subset of $1{,}258$ verified contracts, the fusion classifier attains $89\,\%$ accuracy and a fraud F1-score of 0.67, outperforming GNN-only and LLM-only baselines trained under identical conditions.

The methodological contribution complements the empirical one. The released implementation replaces a single notebook with a multi-file library in which data collection, preprocessing, modelling, training, evaluation, and visualisation are independently testable modules. The two seed arrays are first-class citizens of the codebase, which makes the dataset definition both auditable and extensible.

Four directions for future work follow naturally from the results.

First, the evaluation protocol must be hardened before any deployment claim. As set out in Section VII-F, the random split should be replaced by temporal and deployer-disjoint splits, results should be averaged over multiple seeds with confidence intervals, and the model should be evaluated on a shared public benchmark so that it can be compared directly against TLMG4Eth and BERT4ETH rather than against internal ablations alone.

Second, the fraud F1 ceiling is bounded by the small absolute count of labelled fraud contracts with verified source code. Curating additional fraud labels, either through manual investigation of suspicious Etherscan listings or through automated screening of fresh BigQuery exports, would likely improve fraud recall more than additional architectural tuning.

Third, the language encoder is currently frozen for compute reasons. Lightweight parameter-efficient fine-tuning, for example LoRA adapters, would allow CodeBERT to specialise on Solidity without exceeding the Tesla T4 memory budget.

Fourth, the graph encoder ignores temporal ordering and transaction value, and its node features are limited to degree counts. Replacing GraphSAGE with a temporal graph network that consumes edge timestamps, transaction values, and richer neighbourhood features would let the model distinguish a fraud contract's bootstrapping phase from its run-out phase, which is currently invisible to the static-graph encoder.

The combination of these directions, together with the reproducible pipeline released alongside the paper, provides a base on which the next generation of Ethereum fraud detectors can be built and compared.

Data and Code Availability

The full implementation, including the two seed arrays, the data-collection pipeline, the model definitions, the training and evaluation scripts, and the figure-generation code, is published at the project repository under the `code/` directory. The Forta labelled-datasets release is the canonical source of ground-truth fraud labels and is downloaded automatically at pipeline run time. Transaction data is sourced from Google BigQuery's public Ethereum dataset using the query in `code/data/raw/QUERY.sql`.

Ethical Considerations

This work investigates publicly attributed fraud activity. No private or personally identifiable information is collected. The fusion model is intended as a decision-support tool. Production deployment should combine its predictions with human review to avoid penalising benign addresses that share behavioural signatures with phishing campaigns.

AI Disclosure

The authors used a generative AI coding assistant for refactoring legacy notebook code into a modular library and for copy-editing draft text. All modelling decisions, dataset definitions, experimental results, and the final analysis were produced and verified by the authors.

References

[1] K. Grauer, E. Jardine, E. Leosz, and H. Updegrave, "The 2023 crypto crime report," *Chainalysis Industry Reports*, 2023.

[2] CoinSmart, "Top 5 crypto scams: How to avoid cryptocurrency scams," 2023, accessed: 2024-06-09. [Online]. Available: https://www.coinsmart.com/blog/top-5-crypto-scams/

[3] A. Agrawal, "Cryptocurrency scams: Ponzi and more," 2023, accessed: 2024-06-09. [Online]. Available: https://ashishagrawalcyber.com/scams/cryptocurrency-scams/

[4] DataVisor, "What is a rug pull scam?" 2022, accessed: 2024-06-09. [Online]. Available: https://www.datavisor.com/wiki/rug-pull-scams/

[5] J. Mosakheil and K. Yang, "Decentralized compromise-tolerant public key management ecosystem with threshold validation," 2023. [Online]. Available: https://eprint.iacr.org/2023/1791

[6] M. Drijvers, S. Gorbunov, G. Neven, and H. Wee, "Pixel: Multi-signatures for consensus," Cryptology ePrint Archive, Paper 2019/514, 2019. [Online]. Available: https://eprint.iacr.org/2019/514

[7] Y. Xiao, P. Zhang, and Y. Liu, "Secure and efficient multi-signature schemes for fabric: An enterprise blockchain platform," *arXiv preprint arXiv:2210.10294*, 2022.

[8] Z. Bao, W. Shi, D. He, and K.-K. R. Choo, "Iotchain: A three-tier blockchain-based iot security architecture," *arXiv preprint arXiv:1806.02008*, 2018.

[9] X. Jiang *et al.*, "A2 Chain: A blockchain-based decentralized authentication scheme for 5g-enabled iot," *Mobile Information Systems*, 2020.

[10] M. Ostapowicz and K. Zbikowski, "Detecting fraudulent accounts on blockchain: A supervised approach," in *Web Information Systems Engineering – WISE 2019*, ser. Lecture Notes in Computer Science, vol. 11881. Springer, 2019, pp. 31–46.

[11] J. A. Blanco and A. J. Tallón-Ballesteros, "Supervised machine learning techniques in the bitcoin transactions: A case of ransomware classification," in *SOCO 2021*, ser. Advances in Intelligent Systems and Computing, vol. 1401. Springer, 2022, pp. 803–810.

[12] T. Ashfaq, R. Khalid, A. S. Yahaya, S. Aslam, A. T. Azar, S. Alsafari, and I. A. Hameed, "A machine learning and blockchain based efficient fraud detection mechanism," *Sensors*, vol. 22, no. 19, p. 7162, 2022.

[13] Y. Elmougy and L. Liu, "Demystifying fraudulent transactions and illicit nodes in the Bitcoin network for financial forensics," *arXiv preprint arXiv:2306.06108*, 2023.

[14] J. Kim, S. Lee, Y. Kim, S. Ahn, and S. Cho, "Graph learning-based blockchain phishing account detection with a heterogeneous transaction graph," *Sensors*, vol. 23, no. 1, p. 463, 2023.

[15] J. Zhou, C. Hu, S. Gong, J. Xu, J. Shen, and Q. Xuan, "BlockGC: A joint learning framework for account identity inference on blockchain with graph contrast," *arXiv preprint arXiv:2112.03659*, 2021.

[16] T. Bao *et al.*, "CT-GCN+: A high-performance cryptocurrency transaction graph convolutional model for phishing node classification," *Cybersecurity*, vol. 7, p. 3, 2024.

[17] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics," *arXiv preprint arXiv:1908.02591*, 2019.

[18] H. Huang, X. Zhang, J. Wang, C. Gao, X. Li, R. Zhu, and Q. Ma, "PEAE-GNN: Phishing detection on Ethereum via augmentation ego-graph based on graph neural network," *IEEE Transactions on Computational Social Systems*, vol. 11, no. 3, pp. 4326–4339, 2024.

[19] Z. Chen, S.-Z. Liu, J. Huang, Y.-H. Xiu, H. Zhang, and H.-X. Long, "Ethereum phishing scam detection based on data augmentation method and hybrid graph neural network model," *Sensors*, vol. 24, no. 12, p. 4022, 2024.

[20] M. Li, L. Jia, and X. Su, "Global-local graph attention with cyclic pseudo-labels for Bitcoin anti-money laundering detection," *Scientific Reports*, vol. 15, no. 1, p. 22668, 2025.

[21] S. Li, G. Gou, C. Liu, C. Hou, Z. Li, and G. Xiong, "TTAGN: Temporal transaction aggregation graph network for Ethereum phishing scams detection," in *Proceedings of the ACM Web Conference 2022*, 2022, pp. 661–669.

[22] S. Ouyang, Q. Bai, H. Feng, and B. Hu, "Bitcoin money laundering detection via subgraph contrastive learning," *Entropy*, vol. 26, no. 3, p. 211, 2024.

[23] H. Kanezashi, T. Suzumura, X. Liu, and T. Hirofuchi, "Ethereum fraud detection with heterogeneous graph neural networks," *arXiv preprint arXiv:2203.12363*, 2022.

[24] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, "Making smart contracts smarter," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016, pp. 254–269.

[25] P. Tsankov, A. Dan, D. Drachsler-Cohen, A. Gervais, F. Buenzli, and M. Vechev, "Securify: Practical security analysis of smart contracts," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 67–82.

[26] P. Qian, Z. Liu, Q. He, R. Zimmermann, and X. Wang, "Towards automated reentrancy detection for smart contracts based on sequential models," *IEEE Access*, vol. 8, pp. 19 685–19 695, 2020.

[27] C. F. Torres, M. Steichen, and R. State, "The art of the scam: Demystifying honeypots in Ethereum smart contracts," in *28th USENIX Security Symposium*, 2019, pp. 1591–1607.

[28] M. Zhang, X. Zhang, Y. Zhang, and Z. Lin, "TXSPECTOR: Uncovering attacks in Ethereum from transactions," in *29th USENIX Security Symposium*, 2020, pp. 2775–2792.

[29] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, "GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.

[30] D. Kong, X. Li, and W. Li, "UEChecker: Detecting unchecked external call vulnerabilities in dApps via graph analysis," *arXiv preprint arXiv:2508.01343*, 2025.

[31] S. Hu, Z. Zhang, B. Luo, S. Lu, B. He, and L. Liu, "BERT4ETH: A pre-trained transformer for Ethereum fraud detection," in *Proceedings of the ACM Web Conference 2023*, 2023, pp. 2189–2197.

[32] R. Liang, J. Chen, C. Wu, K. He, Y. Wu, W. Sun, R. Du, Q. Zhao, and Y. Liu, "Towards effective detection of Ponzi schemes on Ethereum with contract runtime behavior graph," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 4, pp. 1–32, 2025.

[33] J. Sun, Y. Jia, Y. Wang, Y. Tian, and S. Zhang, "Ethereum fraud detection via joint transaction language model and graph representation learning," *Information Fusion*, vol. 120, p. 103074, 2025.

[34] C. Wu, J. Chen, Z. Zhao, K. He, G. Xu, Y. Wu, H. Wang, H. Li, Y. Liu, and Y. Xiang, "TokenScout: Early detection of Ethereum scam tokens via temporal graph learning," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 956–970.

[35] L. Galletta and F. Pinelli, "Explainable Ponzi schemes detection on Ethereum," in *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*, 2024, pp. 1014–1023.

[36] R. Camino, C. F. Torres, M. Baden, and R. State, "A data science approach for honeypot detection in Ethereum," *arXiv preprint arXiv:1910.01449*, 2019.

[37] Forta Network, "Labelled datasets: Phishing scams and malicious smart contracts," 2024. [Online]. Available: https://github.com/forta-network/labelled-datasets

[38] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems*, 2017.

[39] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," *arXiv preprint arXiv:2002.08155*, 2020.

[40] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019.